Budget Tracker

Project plan, function hierarchy and reference

Programming Fundamentals, Semester 1 final project
Team of three. One week. One C file. Two CSV files.

What is inside

- **Page 2** — The project at a glance: features, files, the record, the rules, and how to read the star ratings.
- **Pages 3–8** — One hierarchy diagram per menu option: which function calls which, in the order it happens.
- **Page 9** — What the finished screens should look like.
- **Pages 10 onward** — Every function, what it does in one or two sentences, how hard it is, and who else uses it.
- **Last page** — Suggested writing order and the week plan.

Everything in this document matches **`budgetTracker.c`**, which already contains every function as an empty stub with a TODO recipe inside. Pick a function, read its page here, read its TODO, write it, build, test, commit.

The project at a glance

What it does

- Record expenses: date, category, amount, description. Validated at every prompt, nothing can crash it.
- View them as a table, sorted by ID, date or amount, with a category bar chart and a summary underneath.
- Search by category, by month, or by a word in the description.
- Edit one field of an expense, or delete one after a y/n confirmation.
- A monthly budget screen: income, spent, remaining, a spending limit with a green / yellow / red status, and where the money went.
- Monthly income and spending limit are settings the user can change at any time.
- Everything is saved to CSV after every change and loaded back on the next run.

The two files

File	Holds	Written when
`ExpenseFile.csv`	One expense per row. Header: id,day,month,year,category,amount,description	Add, Edit, Delete
`settings.csv`	Two numbers. Header: monthly_income,monthly_limit	Settings

The record

```
struct Expense {
    int    id;
    int    day, month, year;
    int    category;      // 0..7
    double amount;
    char   description[50];
};

Array of 1000 records + a count.
```

```
struct Settings {
    double monthlyIncome;
    double monthlyLimit;   // 0 = no limit
};

Categories (stored 0..7, shown by name):
Food, Transport, Rent, Utilities,
Entertainment, Education, Health, Other
```

Five rules everybody follows

- **Memory is the truth.** Load once at start. Save after add, edit, delete. Never save after sort or search. Sorting works on a copy.
- **Only the four load/save functions touch a file.** Only the six input functions read the keyboard. Everything else works on the array.
- **show prints, calculate computes.** A function does one or the other, never both.
- **Category is stored as 0 to 7** and shown as a name. The user never sees the raw number.
- **IDs are never reused.** Next ID is the biggest existing one plus one, so a deleted 5 never comes back.

Naming rule

`input` asks the user · **`show`** prints · **`is`** yes/no · **`calculate`** maths · **`find`** returns an index · **`load / save`** files

Difficulty stars

☆☆☆☆☆ Trivial, one to three lines.

★☆☆☆☆ Easy, one loop or one if.

★★☆☆☆ A few steps, but every step is known.

★★★☆☆ Needs care: a loop with conditions, formatting, or a new library call.

★★★★☆ The hardest in this project: file parsing, string handling, scaled bars.

★★★★★ Not used. Nothing here is that hard.

Function hierarchy

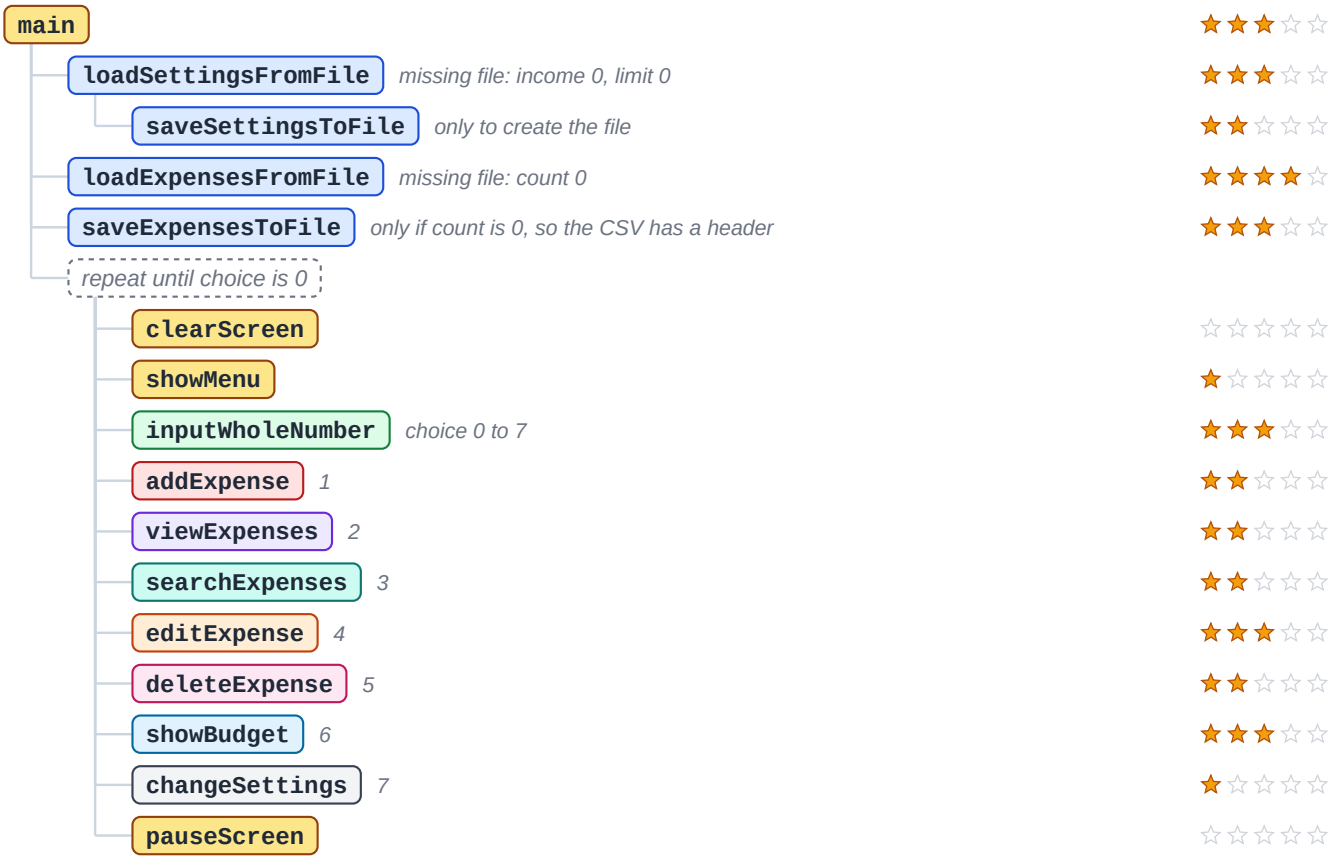
Read each diagram top to bottom: that is the order things happen when the user picks that option. Indentation means *calls*. Grey text beside a box is a note about that call. Stars on the right are difficulty.

MENU FILE INPUT DATE ADD VIEW
SEARCH EDIT DELETE BUDGET SETTINGS

A function is coloured by the group it belongs to in the reference, not by the diagram it appears in. Dashed boxes are not functions, just a step in the flow.

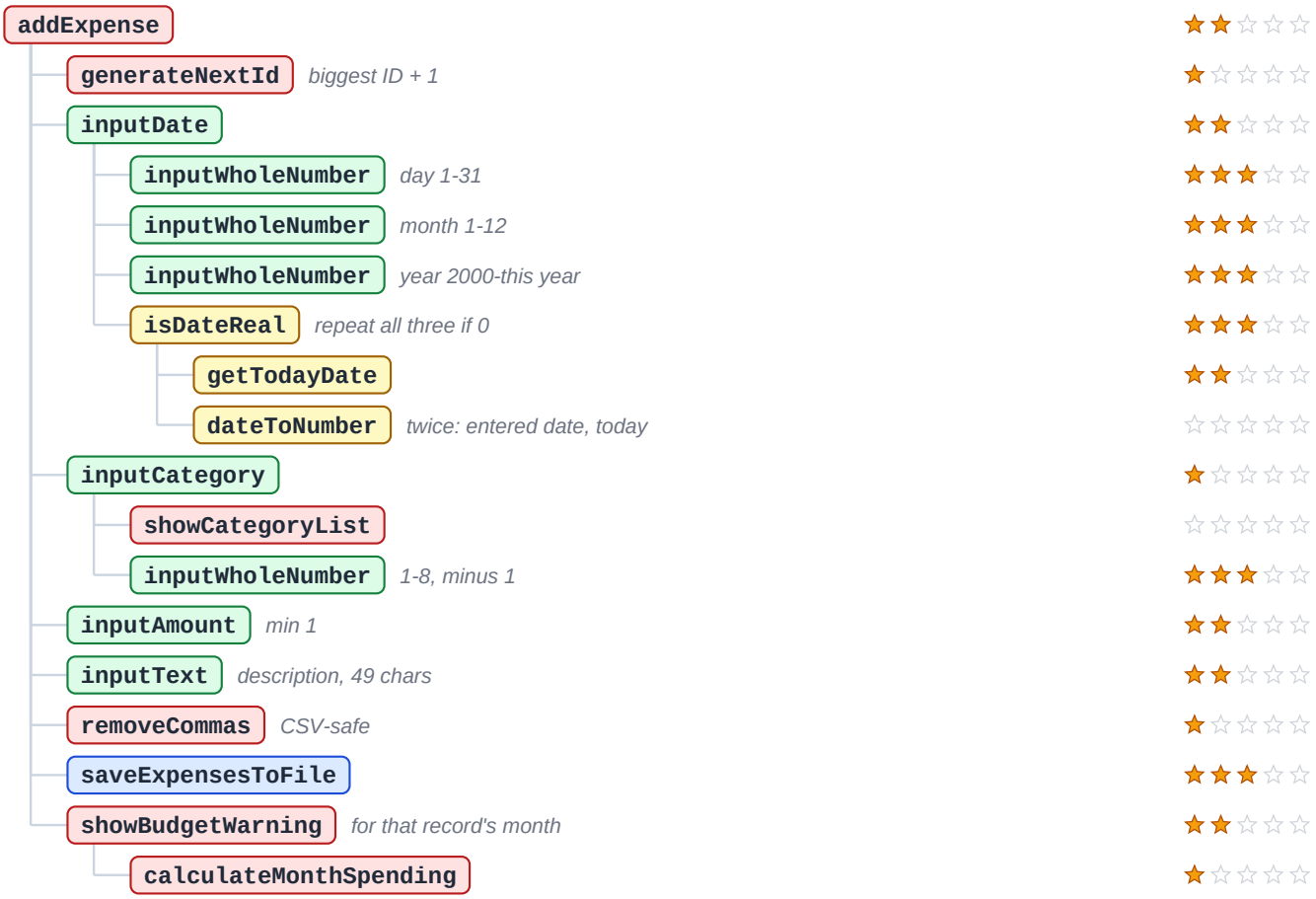
Startup and main menu

What happens when the program starts. Both files are loaded once, then the menu repeats until the user picks 0. Every option is one function call. Add and Delete return the new count.



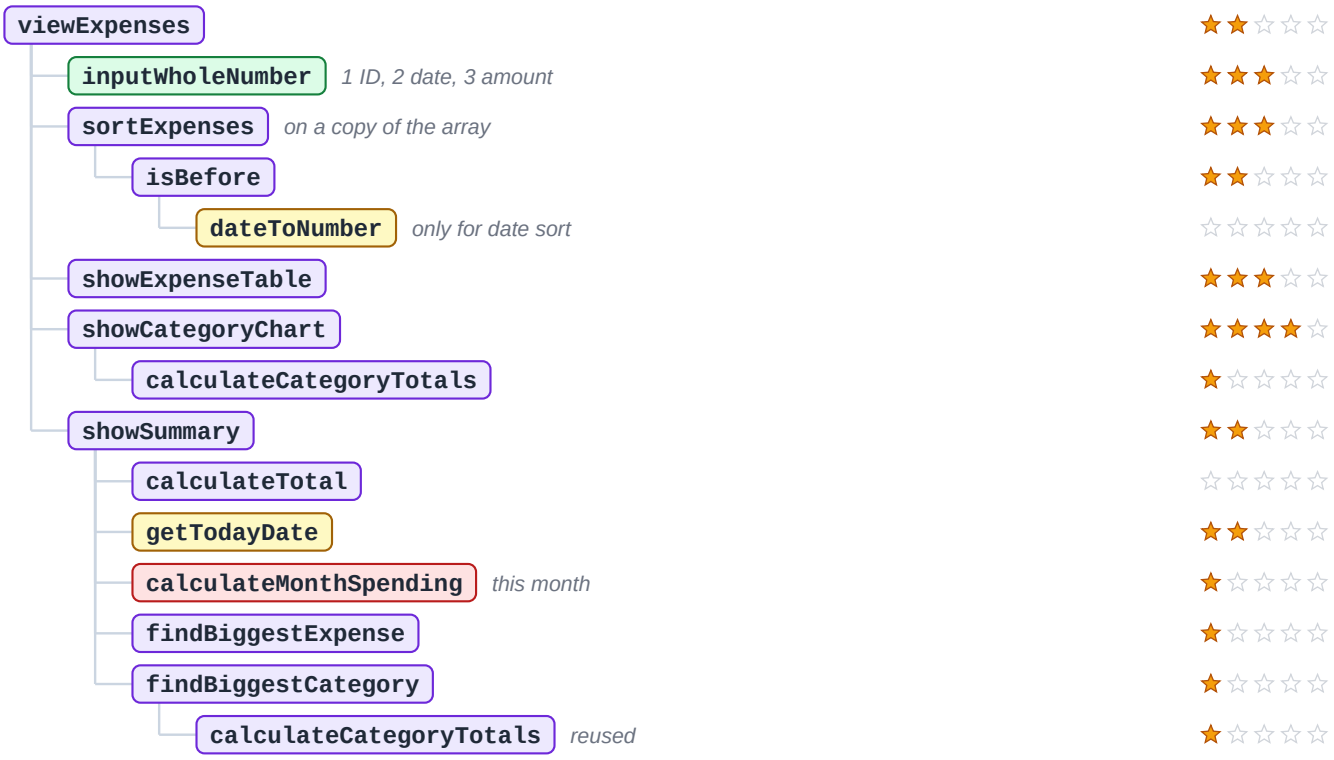
1 Add expense

The option that defines most of the helpers. Seven direct steps; three of them are small trees of their own. Adding always succeeds (unless storage is full). The budget check only warns, it never blocks.



2 View expenses

Sorts a copy of the list, never the real one, so the file is always saved in ID order. show functions only print; calculate and find functions only compute.



3 Search

Three ways to search. Each fills a separate results array and returns how many it found, then the same table function prints it. The total appears only after a month search.

- `searchExpenses` ★★☆☆☆
  - `inputWholeNumber` *1 category, 2 month, 3 description* ★★★☆☆
  - `searchByCategory` ★☆☆☆☆
    - `inputCategory` ★☆☆☆☆
  - `searchByMonth` ★☆☆☆☆
    - `inputWholeNumber` *month* ★★★☆☆
    - `inputWholeNumber` *year* ★★★☆☆
    - `filterByMonth` *shared with Budget* ★☆☆☆☆
  - `searchByDescription` ★★☆☆☆
    - `inputText` *word* ★★☆☆☆
    - `containsIgnoreCase` *once per record* ★★★★☆
  - `showSearchResults` *total flag on for month only* ★☆☆☆☆
    - `showExpenseTable` ★★★☆☆
    - `calculateTotal` *only when flag is on* ☆☆☆☆☆

4 Edit expense

Find it, show it, change one field, save. Every value goes through the same input functions as Add, so Edit has exactly the same validation for free. The ID is the one field that cannot be edited.

- `editExpense` ★★★☆☆
  - `inputWholeNumber` *ID* ★★★☆☆
  - `findById` *-1 means not found: stop* ★☆☆☆☆
  - `showOneExpense` ★☆☆☆☆
  - `inputWholeNumber` *field 1-4, 0 cancel* ★★★☆☆
  - `inputDate` *[1] same checks as Add* ★★☆☆☆
  - `inputCategory` *[2]* ★☆☆☆☆
  - `inputAmount` *[3] min 1* ★★☆☆☆
  - `inputText` *[4]* ★★☆☆☆
    - `removeCommas` ★☆☆☆☆
  - `saveExpensesToFile` ★★★☆☆
  - `showBudgetWarning` *amount or date may have moved* ★★☆☆☆
    - `calculateMonthSpending` ★☆☆☆☆

5 Delete expense

The one place with a y/n confirmation, because a wrong delete cannot be undone. Records after the hole slide left; the ID is never handed out again.

- `deleteExpense` ★★☆☆☆
  - `inputWholeNumber` *ID* ★★★☆☆
  - `findById` *-1 means not found: stop* ★☆☆☆☆
  - `showOneExpense` ★☆☆☆☆
  - `inputYesNo` *n cancels* ★☆☆☆☆
  - `removeAtIndex` *count goes down by one* ★★☆☆☆
  - `saveExpensesToFile` ★★★☆☆

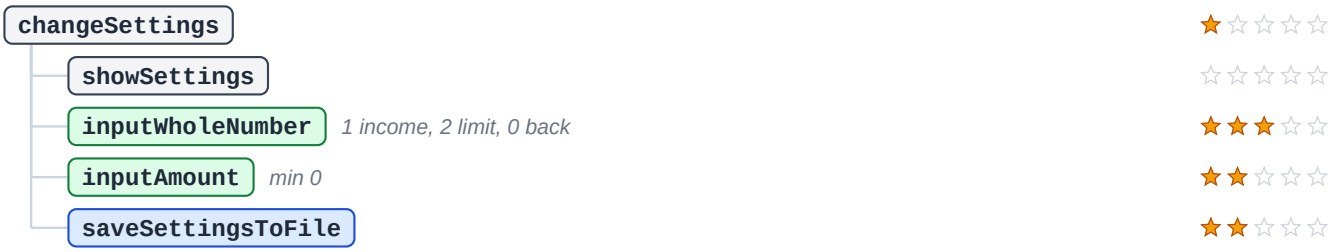
6 Budget screen

Almost no new maths: everything here was written for Add and View. Income minus spent is the remaining, spent against the limit gives the coloured status, and the month's own records feed the chart.

- `showBudget` ★★★☆☆
  - `getTodayDate` *default to this month* ★★☆☆☆
  - `inputWholeNumber` *month* ★★★☆☆
  - `inputWholeNumber` *year* ★★★☆☆
  - `calculateMonthSpending` ★☆☆☆☆
  - `calculateRemaining` *income - spent* ☆☆☆☆☆
  - `showBudgetStatus` *green / yellow / red* ★★☆☆☆
  - `filterByMonth` *that month's records* ★☆☆☆☆
  - `showCategoryChart` *on the filtered records* ★★★★☆
    - `calculateCategoryTotals` ★☆☆☆☆

7 Settings

Monthly income and spending limit live in settings.csv and are changed here. A limit of 0 means no limit, and both warning functions already stay quiet when it is 0.



What the screens should look like

Targets for the show functions. Exact spacing is up to whoever writes them, but this is the level of polish to aim for.

View: table, chart, summary

```
ID   Date        Category          Amount  Description
-------------------------------------------------------------
  1  03/11/2026  Food              450.00  Lunch at cafeteria
  2  05/11/2026  Transport         120.00  Bus card top-up
  3  01/11/2026  Rent           15000.00  November rent
-------------------------------------------------------------

Food           ####################      12,400.00   40%
Rent           ################          10,000.00   32%
Transport      ########                   4,850.00   16%

Total spent (all time)   48,320.00
This month (Nov 2026)    12,450.00
Biggest category         Food            18,900.00
Biggest single expense   #3   Rent       15,000.00  on 01/11/2026
```

Budget

```
================ BUDGET: November 2026 ================
Monthly income        50,000.00
Spent this month      31,250.00
Remaining             18,750.00

Spending limit        30,000.00
Status                OVER BUDGET by 1,250.00        <- red

Where it went:
Food           ############         12,400.00   40%
Rent           ##########           10,000.00   32%
```

Add, with the warning after

```
Day   : 8
Month : 11
Year  : 2026
  1. Food   2. Transport   3. Rent   4. Utilities
  5. Entertainment   6. Education   7. Health   8. Other
Category (1-8): 1
Amount: abc
  Please enter a number of at least 1.00.
Amount: 450
Description: Lunch, with friends
Added #12: 450.00 on Food
Careful: 83% of this month's budget used (24,900.00 of 30,000.00)
```

Function reference

Fifty functions. Most are under ten lines. The *also used by* column shows where a function is reused, which is also a measure of how carefully it should be tested: a bug in inputWholeNumber shows up everywhere.

Menu and startup

Function	What it does	Difficulty	Also used by
`main`	Declares the array, count and settings. Loads both files. Loops: clear, menu, read choice 0 to 7, call the matching function, pause. Add and Delete hand back the new count.	★★★☆☆	-
`showMenu`	Prints the title box and the eight numbered options.	★☆☆☆☆	main
`clearScreen`	Runs the cls command.	☆☆☆☆☆	main
`pauseScreen`	Prints Press Enter to continue and waits for one line.	☆☆☆☆☆	main

Files (the only four that touch the disk)

Function	What it does	Difficulty	Also used by
`loadExpensesFromFile`	Opens ExpenseFile.csv, skips the header, reads every row into the array (7 fields per row, skipping bad rows), returns how many were loaded. Missing file returns 0.	★★★★☆	main
`saveExpensesToFile`	Rewrites ExpenseFile.csv from scratch: header line, then one comma-separated line per record with the amount to 2 decimals.	★★★☆☆	Add, Edit, Delete
`loadSettingsFromFile`	Reads monthly income and spending limit from settings.csv. If the file is missing, sets both to 0 and creates it.	★★★☆☆	main
`saveSettingsToFile`	Writes the header and the two numbers to settings.csv.	★★☆☆☆	Settings

Input (the only six that read the keyboard)

Function	What it does	Difficulty	Also used by
`inputWholeNumber`	Shows a prompt, reads the whole typed line into a string, extracts an integer. Repeats until it gets one between min and max. The most-used function in the program.	★★★☆☆	everywhere
`inputAmount`	Same as inputWholeNumber for a decimal number, with only a minimum. Expenses use min 1, settings use min 0.	★★☆☆☆	Add, Edit, Settings
`inputText`	Reads a full line including spaces straight into the caller's string, cuts the newline, refuses an empty line.	★★☆☆☆	Add, Edit, Search
`inputYesNo`	Asks a question, accepts y/Y for 1 and n/N for 0, repeats on anything else.	★☆☆☆☆	Delete
`inputDate`	Asks day, month, year with inputWholeNumber, then hands them to isDateReal. Repeats all three if the date is not real.	★★☆☆☆	Add, Edit
`inputCategory`	Shows the category list, asks for 1 to 8, returns the answer minus one so it matches the array index 0 to 7.	★☆☆☆☆	Add, Edit, Search

Date helpers

Function	What it does	Difficulty	Also used by
`isDateReal`	Returns 1 only if the day exists in that month (leap years handled), the year is 2000 or later, and the date is not after today.	★★★☆☆	inputDate
`getTodayDate`	Reads the computer clock and fills day, month, year. Fixes the two quirks: months count from 0, years from 1900.	★★☆☆☆	isDateReal, View, Budget
`dateToNumber`	Turns a date into one integer, year x 10000 + month x 100 + day, so 8 Nov 2026 is 20261108 and dates compare with plain < and >.	☆☆☆☆☆	isDateReal, isBefore

Add

Function	What it does	Difficulty	Also used by
`addExpense`	Refuses if storage is full. Builds one record from the input functions, appends it, saves, prints one confirmation line, then shows the budget warning for that month. Returns the new count.	★★☆☆☆	main
`generateNextId`	Scans every record for the biggest ID and returns it plus one. Empty list gives 1. Deleted IDs are never reused.	★☆☆☆☆	addExpense
`removeCommas`	Walks the string and turns every comma into a space, so a description can never break the CSV.	★☆☆☆☆	Add, Edit
`showCategoryList`	Prints the eight category names numbered 1 to 8.	☆☆☆☆☆	inputCategory
`calculateMonthSpending`	Adds up the amount of every record whose month and year match. Returns the total.	★☆☆☆☆	Add, Edit, View, Budget
`showBudgetWarning`	Compares a month's spending to the limit. Prints over-budget, careful at 80%, or a plain status line. Silent if no limit is set.	★★☆☆☆	Add, Edit

View

Function	What it does	Difficulty	Also used by
`viewExpenses`	Asks for a sort order, copies the list, sorts the copy, then prints the table, the category chart and the summary. The real list is never reordered.	★★☆☆☆	main
`sortExpenses`	Bubble sort over the array. Swaps neighbours whenever isBefore says the second should come first. Stops after a pass with no swaps.	★★★☆☆	viewExpenses
`isBefore`	Decides the order of two records for one sort mode: by ID ascending, by date ascending via dateToNumber, or by amount descending.	★★☆☆☆	sortExpenses
`showExpenseTable`	Prints a fixed-width header, a dashed line, one aligned row per record (date as dd/mm/yyyy, category as a name, amount to 2 decimals), and a closing line.	★★★☆☆	View, Search
`calculateCategoryTotals`	Zeroes an array of 8 totals, then adds each record's amount into the slot for its category.	★☆☆☆☆	Chart, findBiggestCategory
`showCategoryChart`	Prints one line per non-empty category: name, a bar of # scaled so the biggest category gets 20, the amount, and its percent of the total.	★★★★☆	View, Budget
`showSummary`	Prints total spent, this month's spending, the biggest category with its total, and the single biggest expense with its details.	★★☆☆☆	viewExpenses
`calculateTotal`	Adds up every record's amount.	☆☆☆☆☆	Summary, Search
`findBiggestExpense`	Returns the index of the record with the largest amount.	★☆☆☆☆	showSummary
`findBiggestCategory`	Uses calculateCategoryTotals and returns the index of the largest total.	★☆☆☆☆	showSummary

Search

Function	What it does	Difficulty	Also used by
`searchExpenses`	Asks category, month or description, runs the matching search into a results array, and shows the results. The total is shown only for a month search.	★★☆☆☆	main
`searchByCategory`	Asks for a category, copies every matching record into results, returns how many.	★☆☆☆☆	searchExpenses
`searchByMonth`	Asks month and year, then hands off to filterByMonth.	★☆☆☆☆	searchExpenses
`filterByMonth`	Copies every record from a given month and year into results, returns how many. Shared with the Budget screen.	★☆☆☆☆	Search, Budget
`searchByDescription`	Asks for a word, copies every record whose description contains it (any case) into results, returns how many.	★★☆☆☆	searchExpenses
`containsIgnoreCase`	Makes lowercase copies of both strings with tolower, then uses strstr to check whether the word appears inside the text.	★★★★☆	searchByDescription
`showSearchResults`	Prints nothing found, or how many were found plus the table, plus the total when asked to.	★☆☆☆☆	searchExpenses

Edit

Function	What it does	Difficulty	Also used by
`editExpense`	Asks for an ID, finds it (stops with a message if not found), shows the record, asks which single field to change, changes it with the same input functions Add uses, saves, re-checks the budget.	★★★☆☆	main
`findById`	Loops over the records and returns the index whose id matches, or -1 if none. Callers must check for -1.	★☆☆☆☆	Edit, Delete
`showOneExpense`	Prints a single record with each field on its own labelled line.	★☆☆☆☆	Edit, Delete

Delete

Function	What it does	Difficulty	Also used by
`deleteExpense`	Asks for an ID, finds it, shows it, asks y/n, then removes it, saves, and returns the new count. Cancels cleanly on n or a bad ID.	★★☆☆☆	main
`removeAtIndex`	Slides every record after the given index one slot to the left, closing the gap. The caller lowers count by one.	★★☆☆☆	deleteExpense

Budget

Function	What it does	Difficulty	Also used by
`showBudget`	Asks for a month (suggesting the current one), then prints income, spent, remaining, the limit, the coloured status, and a category chart for just that month.	★★★☆☆	main
`calculateRemaining`	Returns monthly income minus spent. May be negative.	☆☆☆☆☆	showBudget
`showBudgetStatus`	Prints OK in green, CAREFUL at 80% in yellow, or OVER BUDGET by X in red, using ANSI colour codes. Says no limit set when the limit is 0.	★★☆☆☆	showBudget

Settings

Function	What it does	Difficulty	Also used by
`changeSettings`	Shows the current numbers, asks which to change, reads the new value with inputAmount (min 0), saves the file.	★☆☆☆☆	main
`showSettings`	Prints monthly income and spending limit, or none when the limit is 0.	☆☆☆☆☆	changeSettings

Suggested writing order

Most reused and most foundational first. Each stage can be tested on its own before the next begins.

Stage	Functions	Done when
1 Foundation	**`inputWholeNumber, inputAmount, inputText, inputYesNo, dateToNumber, getTodayDate, isDateReal, inputDate, showCategoryList, inputCategory, showMenu, clearScreen, pauseScreen`**	Test: a menu that loops, refuses letters, and quits on 0. A date prompt that rejects 31 Feb and tomorrow.
2 Files	**`saveExpensesToFile, loadExpensesFromFile, saveSettingsToFile, loadSettingsFromFile`**	Test: hand-type a CSV with two rows, load it, print the records. Delete the file, run again, no crash.
3 Add	**`generateNextId, removeCommas, calculateMonthSpending, showBudgetWarning, addExpense`**	Test: add three expenses, restart, they are still there. IDs are 1, 2, 3.
4 View	**`showExpenseTable, calculateCategoryTotals, showCategoryChart, calculateTotal, findBiggestExpense, findBiggestCategory, showSummary, isBefore, sortExpenses, viewExpenses`**	Test: table aligned, bars scale, sort by amount puts the biggest first, file still in ID order afterwards.
5 Search, Edit, Delete	**`findById, showOneExpense, filterByMonth, searchByCategory, searchByMonth, containsIgnoreCase, searchByDescription, showSearchResults, searchExpenses, editExpense, removeAtIndex, deleteExpense`**	Test: edit an amount and see the warning change. Delete #2, add one, new ID is 4 not 2. Search 'LUNCH' finds 'lunch'.
6 Budget, Settings	**`calculateRemaining, showBudgetStatus, showBudget, showSettings, changeSettings`**	Test: set income 50000 and limit 30000, add until over, status turns red. Set limit 0, warnings disappear.
7 Polish	**`colours, title box, alignment, comments at the top of every function, README, rehearse the demo twice`**	Test: everyone tries to crash it for twenty minutes. Nobody can.

The week

Day	Goal	Done when
Day 1	Skeleton	budgetTracker.c compiles with every stub. First commit. Everyone reads this PDF.
Day 2	Hand out	Each member picks functions from stages 1 to 3. First real functions committed.
Day 3-4	Build	Stages 1 to 4 done. Add and View work end to end with the real file.
Day 5	Integrate	Stages 5 and 6 done. Every menu option works. Restart keeps the data.
Day 6	Break it	Letters in number fields, empty file, missing file, 0 amount, commas, 1000 records, 31 Feb, a future date.
Day 7	Polish	Colours, alignment, comments, README. Rehearse the demo twice.

Git, every time, in this order

```
git pull
...edit only your own functions, build, test...
git add budgetTracker.c
git commit -m "add: generateNextId and removeCommas"
git pull
git push
```

Commit small and often. If the second pull reports a conflict, two people edited the same function. Talk, then fix.